

Techniques Web en python

Cours 1 : présentation du programme du semestre et rappels



Objectifs du semestre

- Consolider les acquis de 1ère année et développer des réflexes
- Approfondir sa maîtrise de python sur des points plus techniques (faits de syntaxe « idiomatiques », notions avancées comme la programmation asynchrone...)
- Se préparer aux entretiens techniques pour trouver un stage/job
- Devenir autonome sur des projets plus complexes, savoir identifier de quelle ressource on a besoin face à un problème pratique et se l'approprier
- Mieux comprendre le Web et savoir l'utiliser (API, front end...)

Structure de chaque cours

- Pratique autour d'un sujet d'interview technique corrigé ensemble (30 mn)
- Présentation d'un point synthétique sur une notion de programmation (1h)
- Application pratique dans un TD

Programme indicatif

- S1 Rappel sur les fichiers
- S2 Python : syntaxe idiomatique et raccourcis
- Sx Crawler le web
- Sy Les API

Autres thèmes : la programmation asynchrone, la récursivité, les décorateurs, utiliser python comme un shell, monitorer ses programmes, pydantic...

Modalités d'évaluation

- 1 ou 2 contrôles alliant connaissances du cours et programmation
- 1 ou 2 TP notés, au hasard parmi les 4

Les TPS

- Faits pour être traités sur plusieurs séances, illustrent les points de cours vus
- Invitent à l'autonomie et à s'appropriier des concepts.
- Doivent être rendus à temps (car je donne des corrigés...) sinon 0.

A priori 4 :

- TP1 : De Flaubert à FlauBERT : révisions et teasers
- TP2 : une histoire de papyrus : collecter et analyser des données
- TP3 : la face cachée de l'INALCO : dorking et secrets d'internet...
- TP4 : Les API en folies

Interview

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have **exactly one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4]`, `target = 6`

Output: `[1,2]`

Interview :

De quels éléments de programmation avons nous besoin ?

Quelles structures de données utiliser ?

1 ère solution naïve

- On teste toutes les possibilités. Quelle est la complexité ?
- Technique de « brute force » mais ça marche. On peut faire mieux...

```
def twoSum(self, nums: List[int], target: int) -> List[int]:  
    for i in range(len(nums)):  
        for j in range(i + 1, len(nums)):  
            if nums[j] == target - nums[i]:  
                return [i, j]  
    # Return an empty list if no solution is found  
    return []
```

2 : trick

On cherche a et b dans la liste tels que $a+b = \text{result}$

Donc pour chaque élément a de la liste il faut voir si $c-a$ est dans la liste aussi et renvoyer son indice. Quelle est la nouvelle complexité ? Quel est l'inconvénient de cette méthode

```
def twoSum(self, nums: List[int], target: int) -> List[int]:  
    num_to_index = {}  
    for i in range(len(nums)):  
        num_to_index[nums[i]] = i  
    for i in range(len(nums)):  
        complement = target - nums[i]  
        if complement in num_to_index and num_to_index[complement] != i:  
            return [i, num_to_index[complement]]  
    # If no valid pair is found, return an empty list  
    return []
```

3) optimisation de la mémoire

- NB : on peut améliorer ce code en utilisant la méthode `list.index(objet)` de python. On peut aussi utiliser le pythonisme « `enumerate` »

```
def twoSum(self, nums: List[int], target: int) -> List[int]:  
    num_to_index = {}  
    for i in range(len(nums)):  
        complement = target - nums[i]  
        if complement in num_to_index:  
            return [i, num_to_index[complement]]  
        num_to_index[nums[i]] = i  
    # Return an empty list if no solution is found  
    return []
```

Qu'est ce qu'un fichier ?

La gestion des fichiers :

Un fichier en informatique est **une unité de stockage de données** utilisée pour conserver et organiser des informations sur un support numérique (disque dur, clé USB, etc.). Il peut contenir différents types de données, comme du texte, des images, des vidéos, ou des programmes exécutables. Chaque fichier a un nom et une extension (ex. : .txt, .jpg, .exe), qui indique son type et permet au système d'exploitation de savoir comment l'ouvrir ou le traiter.

Un fichier est un ensemble de données (bits) stockés quelque part

Le fichier tel qu'on le voit sur son bureau est une illusion, un raccourci vers un espace mémoire.

Les différents types de fichiers

- Source : wikipedia
- Le format d'un fichier est une **convention**
ex : le json

Exécutables	Fichiers qui peuvent être exécutés par l'ordinateur — autrement dit, des programmes .	.exe, .com, .sh, .bat, etc.
Compressés	Fichiers codés selon un procédé qui les rend plus petits que les fichiers originaux non codés. Un programme <i>décompresseur</i> est nécessaire pour effectuer le codage inverse et retrouver ainsi le fichier original.	.arc, .zip, .rar, .z, .arj, .sit, .gz, .7z, etc.
Images	Fichiers qui contiennent des images et du son sous une forme exploitable par l'ordinateur. De tels fichiers peuvent contenir des photos, des pictogrammes, des graphiques, des chansons, de la musique, des émissions radio ou des films.	.gif, .jpg, .bmp, .png, .eps, .tif, etc.
Audio		.au, .wav, .mp3, .oga, .ram, etc.
Vidéo		.avi, .mpg, .mov, ...
Documents	Documents écrits, destinés à être imprimés et lus. Le fichier contient le texte ainsi que les informations de typographie (polices de caractères , couleurs).	.docx, .odt, .html, .doc, etc.
Texte	Fichiers texte <i>brut</i> qui contiennent des textes écrits, sans indications de typographie. Il peut s'agir de textes destinés aux utilisateurs, tels que des modes d'emploi ou des brouillons ; ou alors de textes destinés à l'ordinateur tels que du code source ou bien des données pour un programme.	.txt, .html, .ini, .csv, .log, .conf, .c, etc.

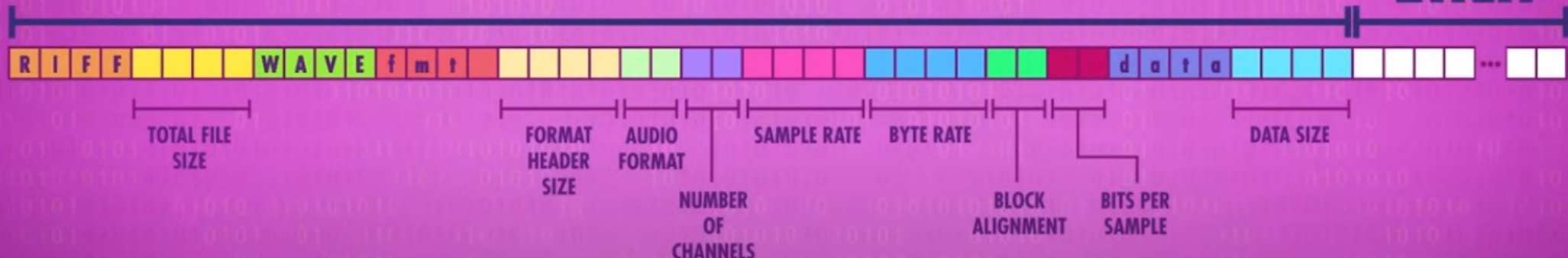
Qu'y a t il dans un fichier ?

- <https://www.youtube.com/watch?v=KN8YqJnShPM>



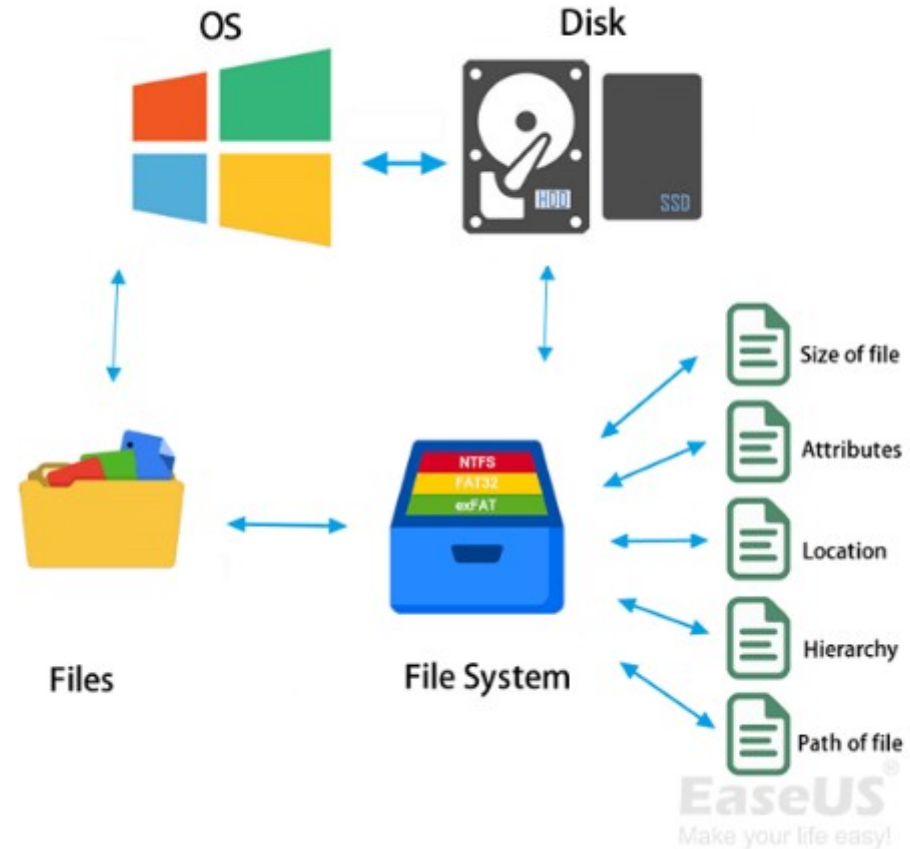
METADATA

DATA



Le file system

- C'est une abstraction créée par le système d'exploitation
- En python on peut se servir des librairies `os` et `pathlib` pour simuler les actions sur le système de fichier comme depuis le shell



Manipuler un fichier c'est donc manipuler une suite de 0 et de 1, de bytes.

Pour manipuler un fichier en python il faut donc charger les données qui sont à l'intérieur

Plusieurs représentations

Ne pas confondre :

- Le path : chemin d'accès au fichier dans l'arborescence (str ou objet de type Path (pathlib))
- Les données elles-mêmes ou flux (série de bytes, inexploitable en soi)
- La donnée bien interprétée dans son format (objet image, chaîne de caractère...) et qu'on peut modifier avec son programme.

=> certaines fonctions attendent une de ces 3 formes et il ne faut pas se tromper

Exemple : les json

Path : Exemple : `"/home/user/documents/file.json"` ou
`Path("/home/user/documents/file.json")`

- flux de données : Exemple : `b'{"name": "Alice", "age": 30, "prof": false}'`
- Objet python : soit objet de classe `json` et on accède à ses champs avec un `obj.field` Et non `obj[«field »]`
- soit converti en dictionnaire : `{"name": "Alice", "age": 30, "prof": false}`

Functions and How They Move Between Forms:

1. `json.load(f)`:

This function **deserializes** JSON content from a **file-like object** into a Python object.

- **Input:** File object or file stream (typically opened in text mode like `'r'`).
- **Output:** Python object (such as a dictionary, list, etc.).

Example:

```
python Copier le code  
  
import json  
  
with open('data.json', 'r') as f:  
    data = json.load(f) # Reads the file and converts it to a Python object (d  
    print(data) # {'name': 'Alice', 'age': 30}
```

2. `json.loads(s)`:

This function **deserializes** a JSON-encoded **string** into a Python object.

- **Input:** JSON string (`str`).
- **Output:** Python object (like a dictionary, list, etc.).

Example:

python

 Copier le code

```
json_str = '{"name": "Alice", "age": 30}'  
data = json.loads(json_str) # Converts the string to a Python object  
print(data) # {'name': 'Alice', 'age': 30}
```

3. `json.dump(obj, f)`:

This function **serializes** a Python object into a JSON **formatted string** and writes it to a **file-like object** (such as a file opened for writing).

- **Input:** Python object (dict, list, etc.), file object.
- **Output:** Writes JSON-encoded content to the file.

Example:

```
python Copier le code  
  
data = {"name": "Alice", "age": 30}  
with open('data.json', 'w') as f:  
    json.dump(data, f) # Serializes the Python object and writes it as JSON to
```

4. `json.dumps(obj)` :

This function **serializes** a Python object into a JSON **formatted string**.

- **Input:** Python object (like dict, list, etc.).
- **Output:** JSON string.

Example:

<

python

 Copier le code

```
data = {"name": "Alice", "age": 30}
json_str = json.dumps(data) # Converts the Python object to a JSON string
print(json_str) # '{"name": "Alice", "age": 30}'
```

Function	Input	Output	Purpose
<code>json.load(f)</code>	File object (<code>f</code>) with JSON content)	Python object	Reads JSON from a file and converts it to a Python object
<code>json.loads(s)</code>	JSON string (<code>s</code>)	Python object	Converts a JSON string to a Python object
<code>json.dump(obj, f)</code>	Python object (<code>obj</code>), file object (<code>f</code>)	Writes JSON content to a file	Serializes a Python object to a JSON file
<code>json.dumps(obj)</code>	Python object (<code>obj</code>)	JSON string	Converts a Python object to a JSON string
File to String			
<code>f.read()</code>	File object (<code>f</code>) with JSON content)	JSON string	Reads entire file as a string
File to Bytes			
<code>f.read()</code>	File object (opened in <code>'rb'</code> mode)	Bytes	Reads raw bytes from a binary file
<code>json.load(BytesIO(bytes))</code>	Bytes-like object	Python object	Reads bytes containing JSON content, converts to Python object

Pipeline classique :

1) charger le fichier depuis la mémoire en une suite de bytes :

with open('data.json', 'r') as f:

- Bonne pratique : enclore l'ouverture du flux dans la balise « with : »

=> permet une bonne gestion de la mémoire et la libère immédiatement après utilisation de l'objet

=> ferme le fichier automatiquement

=> gère mieux les erreurs pour préserver l'intégrité du fichier

Pipeline classique :

2) transformer ce flux dans le format qui nous intéresse, si besoin avec une librairie spécifique selon le format de notre fichier.

with open('file.txt', 'r') as file:

- `content = file.read()`
- Ou ; `reader = csv.reader(file)`
- Ou : `data = json.load(file)`

NB : les fichiers en binaires se lisent avec « rb »

- with open('image.png', 'rb') as file:
- `content = file.read()`

Pipeline classique

- Pour gérer le bon rangement : os et pathlib

3) pour réécriture dans un fichier : inverse, on passe du flux de données aux bytes stockés dans le fichier

with open('output.txt', 'w') as file:

```
    file.write("Hello, World!")
```

Traiter les fichiers avec python

=> des fonctions natives permettent de traiter des fichiers comme en C, avec les opérations basiques de lecture ou écriture (bas niveau)

=> des librairies permettent le même niveau d'abstraction qu'on a avec le shell et traitent les fichiers comme le système d'exploitation <https://realpython.com/working-with-files-in-python/>

=> des librairies abstraient le système de fichiers du stockage. C'est le principe qui a rendu possible le cloud (Amazon S3...)

Conclusion

- Quand vous manipulez des fichiers, comprenez bien quel objet vous manipulez dans votre code
- Quand vous n'arrivez pas à faire fonctionner une fonction qui manipule un fichier, c'est peut être que vous lui avez donné la mauvaise représentation du fichier. Lisez bien la documentation
- Et le web dans tout ça ?
Le web n'est après tout qu'un ramassis de fichiers stockés sur plusieurs machines... la suite aux prochains épisodes.